

Bluetooth Communication

I. Experiment Objective

Learn the BLE (Bluetooth Low Energy) communication function of the MicroROS-V2 control board. Through this experiment, you will master how to initialize a BLE GATT server, create an SPP service with characteristic values, and implement data transmission and reception with a phone or other BLE devices.

II. Experiment Principle

1. Keywords

Keyword	Description
BLE (Bluetooth Low Energy)	Bluetooth Low Energy technology, designed for low-power devices. It supports short-range data communication and is widely used in IoT devices.
GATT (Generic Attribute Profile)	The Generic Attribute Protocol, which defines the architecture of data communication between BLE devices, based on a client-server model.
SPP (Serial Port Profile)	Serial Port Profile, a service mode that emulates traditional Bluetooth serial communication over BLE.
Characteristic	The data unit in a GATT service, containing a value, properties, and descriptors. It is the actual carrier of data reads and writes.
UUID	Universally Unique Identifier, used to uniquely identify BLE services and characteristic values (16-bit/128-bit).
Advertising	A BLE device periodically sends advertising packets to announce its presence so that peripheral devices can discover and connect to it.

2. Technical Architecture

This example implements a BLE GATT server based on the ESP32 Bluedroid protocol stack. After startup, the program initializes NVS Flash, the Bluetooth controller, and the Bluedroid protocol stack, registers a GATT application, and creates an SPP service (UUID 0xFFE0) with 4 characteristic values for data transmission and reception. The device advertises under the name "YahBoom_BL" and waits for a phone or other BLE client to connect.

Main component files:

- `main/main.c` — Program entry, initializes BLE
- `components/ble_server/ble_server.c` — BLE GATT service implementation
- `components/ble_server/ble_server.h` — Characteristic index and constant definitions

III. Experiment Steps

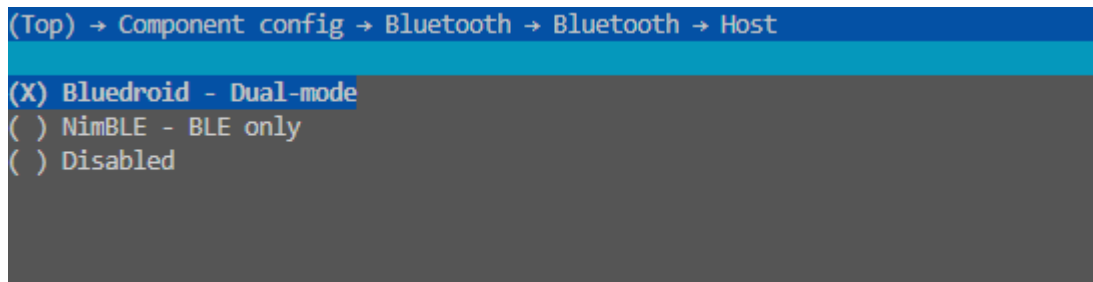
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	Required
USB data cable	Required
A phone that supports BLE (with nRF Connect or another Bluetooth debugging tool installed)	Required

2. Prerequisites

1. Set up the ESP32-IDF development environment
2. Connect the MicroROS-V2 control board to the computer via USB
3. First, the Bluetooth feature is disabled by default in ESP-IDF, so you need to enable Bluetooth in the configuration file.

```
cd ~/esp/base_samples/bluetooch
idf.py menuconfig
```



Because only the Bluetooth BLE 5.0 feature is enabled by default, to make the compilation pass, you need to manually enable the Bluetooth BLE 4.2 feature.

```
(Top) → Component config → Bluetooth → Bluedroid Options
Espressif IoT Development Framework Configuration

(3072) Bluetooth event (callback to application) task stack size (NEW)
  The cpu core which Bluedroid run (Core 0 (PRO CPU)) --->
(4096) Bluetooth Bluedroid Host Stack task stack size (NEW)
[ ] Bluedroid memory debug (NEW)
[*] Bluetooth Low Energy (NEW)
[*]   Include GATT server module(GATTS) (NEW)
[ ]     Enable Peripheral Preferred Connection Parameters characteristic in GAP service (NEW)
[ ]     Include blufi function (NEW)
(8)   Max GATT Server Profiles (NEW)
(100) Max GATT Service Attributes (NEW)
      GATTS Service Change Mode (GATTS automatically send service change indication) --->
[ ]   Enable Robust Caching on Server Side (NEW)
[ ]   Allow to write device name by GATT clients (NEW)
[ ]   Allow to write appearance by GATT clients (NEW)
[*]   Include GATT client module(GATT_C) (NEW)
(40)   Max gattc cache characteristic for discover (NEW)
[ ]     Save gattc cache data to nvs flash (NEW)
(3)     The number of attempts to reconnect if the connection establishment failed (NEW)
[*]   Include BLE security module(SMP) (NEW)
[ ]     Slave enable connection parameters update during pairing (NEW)
[ ]   Disable BT debug logs (minimize bin size) (NEW)
      BT_DEBUG_LOG_LEVEL --->
(4)   BT/BLE MAX_ACL_CONNECTIONS(1~9) (NEW)
[*]   Enable BLE multi-connections (NEW)
[ ]   BT/BLE will first malloc the memory from the PSRAM (NEW)
[ ]   Use dynamic memory allocation in BT/BLE stack (NEW)
[ ]   BLE queue congestion check (NEW)
[ ]   Report adv data and scan response individually when BLE active scan (NEW)
(30) Timeout of BLE connection establishment (NEW)
(32) length of bluetooth device name (NEW)
(900) Timeout of resolvable private address (NEW)
[*]   Enable BLE 5.0 features (NEW)
[*]   Enable BLE 4.2 features
[ ]   Enable BLE high duty advertising interval feature (NEW)
```

3. Compile and Flash

```
cd ~/esp/base_samples/bluetooth
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

4. Stopping Method

In the terminal, press `Ctrl+]` to exit the serial monitor.

IV. Experiment Result

After successful compilation and flashing, the Monitor will output:

```
hello yahboom
I (xxx) MAIN: Nice to meet you!
I (xxx) BLE_SERVER: BLE_Init init bluetooth
```

Using a BLE debugging tool such as nRF Connect or LightBlue on your phone, you can scan for the BLE device named **YahBoom_BL**. After connecting, you can see the service with UUID `0xFFE0` and its 4 characteristic values underneath. You can write data to the `0xFFE1` characteristic, and the control board will print the received data in the log.

V. Program Analysis

Source code paths:

- Firmware source: `~/esp/base_samples/bluetooth/main/main.c`
- BLE service: `~/esp/base_samples/bluetooth/components/ble_server/ble_server.h`,
`ble_server.c`

1. Main Program Logic

`main/main.c` is the program entry. After printing a welcome message, it calls `BLE_Init()` to initialize the BLE protocol stack:

```
#include "ble_server.h"

void app_main(void)
{
    printf("hello yahboom\n");
    ESP_LOGI(TAG, "Nice to meet you!");
    BLE_Init();
}
```

2. BLE Service Initialization Flow

The `BLE_Init()` function in `components/ble_server/ble_server.c` completes the full initialization of the BLE protocol stack with the following steps:

1. **Initialize NVS Flash** — Call `nvs_flash_init()` to initialize non-volatile storage for saving Bluetooth configuration information. If the NVS partition is abnormal, erase it first and then initialize.
2. **Release classic Bluetooth memory** — Call `esp_bt_controller_mem_release(ESP_BT_MODE_CLASSIC_BT)` to release the memory occupied by classic Bluetooth, keeping only the BLE function.
3. **Initialize the Bluetooth controller** — Start the BLE controller through `esp_bt_controller_init()` and `esp_bt_controller_enable(ESP_BT_MODE_BLE)`.
4. **Initialize Bluedroid** — Call `esp_bluedroid_init()` and `esp_bluedroid_enable()` to start the Bluedroid protocol stack.
5. **Register callback functions** — Register the GATT Server callback and the GAP callback.
6. **Register the application** — Call `esp_ble_gatts_app_register(ESP_SPP_APP_ID)` to register the SPP application, where `ESP_SPP_APP_ID` is defined as `0x56`.

3. BLE Device Name and Advertising Data

The device advertising name is `YahBoom_BL`, and the advertising data packet format is as follows:

```
static const uint8_t spp_adv_data[] = {
    0x02, 0x01, 0x06,                /* Flags */
    0x03, 0x03, 0xF0, 0xAB,          /* 16-bit Service Class UUIDs */
    0x0B, 0x09, 'Y', 'a', 'h', 'B', 'o', 'o', 'm', '_', 'B', 'L' /* Complete Local Name */
};
```

Advertising parameter settings:

```
static esp_ble_adv_params_t spp_adv_params = {
    .adv_int_min = 0x20,             // Minimum advertising interval
    .adv_int_max = 0x40,             // Maximum advertising interval
    .adv_type = ADV_TYPE_IND,        // Connectable advertising type
    .own_addr_type = BLE_ADDR_TYPE_PUBLIC,
    .channel_map = ADV_CHNL_ALL,
    .adv_filter_policy = ADV_FILTER_ALLOW_SCAN_ANY_CON_ANY,
};
```

4. SPP Service and Characteristic Values

This example implements a BLE service similar to SPP (Serial Port Profile). The service UUID is `0xFFE0` and it contains 4 characteristic values:

Characteristic	UUID	Properties	Description
SPP_DATA_RECEIVE	0xFFE1	Read/Write	Data reception channel, used to receive data sent from the phone
SPP_DATA_NOTIFY	0xFFE2	Read/Notify	Data notification channel, used to send data notifications to the phone
SPP_COMMAND_RECEIVE	0xFFE3	Read/Write	Command reception channel, used to receive commands sent from the phone
SPP_COMMAND_NOTIFY	0xFFE4	Read/Notify	Command status notification channel, used to notify the phone of status changes

The data length limits of each characteristic (defined in `ble_server.h`):

- `SPP_DATA_MAX_LEN` = 512 bytes (maximum length of the data channel)
- `SPP_CMD_MAX_LEN` = 20 bytes (maximum length of the command channel)
- `SPP_STATUS_MAX_LEN` = 20 bytes (maximum length of the status channel)

5. Data Reception and Processing

The `gatts_profile_event_handler` handles the `ESP_GATTS_WRITE_EVT` event:

- When the write handle is `SPP_IDX_SPP_DATA_RECV_VAL`, call `BLE_Parse_Rx_Data()` to process the received data
- When the write handle is `SPP_IDX_SPP_COMMAND_VAL`, save the command data to the buffer

- When the write handle is `SPP_IDX_SPP_DATA_NTF_CFG`, enable or disable the notification function according to the CCCD value written by the client

6. Key Components

`ble_server.h` defines the characteristic index enum and key constants:

Parameter definition file: `~/esp/base_samples/bluetooth/components/ble_server/ble_server.h`

Definition	Value	Description
<code>SPP_IDX_SVC</code>	0	Service declaration index
<code>SPP_IDX_SPP_DATA_RECV_VAL</code>	2	Data reception characteristic index
<code>SPP_IDX_SPP_DATA_NTY_VAL</code>	4	Data notification characteristic index
<code>SPP_IDX_SPP_COMMAND_VAL</code>	7	Command characteristic index
<code>SPP_IDX_SPP_STATUS_VAL</code>	9	Status characteristic index
<code>SPP_IDX_NB</code>	11	Total number of attribute table entries

7. Connection and Disconnection Events

- **Connection event (`ESP_GATTS_CONNECT_EVT`):** Save the connection ID and the GATT interface handle, and set `is_connected = true`
- **Disconnection event (`ESP_GATTS_DISCONNECT_EVT`):** Reset the connection state and restart advertising to wait for a new connection
- **MTU negotiation (`ESP_GATTS_MTU_EVT`):** Save the negotiated MTU size to `spp_mtu_size`, with an initial value of 23 bytes

VI. Common Problems

Problem	Cause	Solution
Cannot scan the YahBoom_BL device	Advertising did not start normally or the controller is faulty	Check the Monitor log and confirm that BLE initialization succeeded
Cannot connect to the BLE device	Bluetooth compatibility issue with the phone or computer	Try using a different BLE debugging tool
NVS initialization fails	The NVS partition data is corrupted	Run <code>idf.py erase_flash</code> to erase and re-flash